



Figure 9: Accelerometer

```
private TextView mSensorValuesUp;
private TextView mSensorValuesDown;
private TextView mSensorValuesZUp;
private TextView mSensorValuesZDown;
```

Next, reference them in your Java class; this can be done with the following code. (These basic concepts were covered in my article in LFY's October 2010 edition.)

```
/** Called when the activity is first created. */
@Override
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.main);

    mSensorValuesLeft = (TextView) findViewById(R.
id.txtSensorValueLeft);
    mSensorValuesRight = (TextView) findViewById(R.
id.txtSensorValueRight);
    mSensorValuesUp = (TextView) findViewById(R.
id.txtSensorValueUp);
    mSensorValuesDown = (TextView) findViewById(R.
id.txtSensorValueDown);
    mSensorValuesZUp = (TextView) findViewById(R.
id.txtSensorZValueUp);
    mSensorValuesZDown = (TextView) findViewById(R.
id.txtSensorZValueDown);
```

The basic classes used to access sensors are: *SensorManager*, *Sensor*, *SensorListener* and *SensorEvent*. The *SensorManager* class provides an app the ability to access sensors and interact with them. The *Sensor* class is the model class for sensors, and provides the basic API to get values from the underlying sensor. The *SensorListener* class is a listener class (like a button *onClick* listener). It allows the app to register itself with the listener, to receive notifications of changes on the sensors, with the new

values. The *SensorEvent* class interacts with the sensors, and has some basic methods that access sensors and retrieve data in case of any event or change.

Sensors are provided by the system service, so first claim the sensor service from the system service with the following code:

```
mSensorManager = (SensorManager)
getSystemService(Context.SENSOR_SERVICE);
```

Now go ahead and use the above to implement your app:

```
mAccelerometer = mSensorManager.getDefaultSensor(Sensor.TYPE_
ACCELEROMETER);
mSensorManager.registerListener(this, mAccelerometer,
SensorManager.SENSOR_DELAY_UI);
```

Start by getting hold of the sensor using the *getDefaultSensor()* method, which returns the default sensor for the type passed as a parameter. You have requested *Sensor.TYPE_ACCELEROMETER*. After getting the sensor, register your app with the sensor listener, to be able to listen to changes detected by the sensor, with *RegisterListener()*. *SensorManager.SENSOR_DELAY_UI* represents the delay in listening to sensor events. This constant specifies listening at speeds normal for a user interface, because continuous listening to sensors drains the battery very quickly. Other constants are *SensorManager.SENSOR_DELAY_FASTEST*, *SensorManager.SENSOR_DELAY_NORMAL*, and *SensorManager.SENSOR_DELAY_GAME*, whose purposes are obvious.

```
@Override
public void onAccuracyChanged(Sensor sensor, int accuracy) {
    /*
     * You could have values defined for x, y and z axes and
     add these
     * values to the original values while using it in your
     app
     */
}

@Override
public void onSensorChanged(SensorEvent event) {
}
}
```

There are two methods that need to be overridden: *onAccuracyChanged()* and *onSensorChanged()*. The first is used to calibrate the sensor if required (you implement it if you want a custom calibration, not the default). *OnSensorChanged()* is invoked every time a change is